

@inandu-solutions/grid-angular

User manual · v1.0.0

1. Introduction
2. Installation
3. Quick start
4. Data & rows
5. Columns
6. Cell types & formatting
7. Sorting
8. Filtering
9. Pagination
10. Virtual scroll
11. Server-side mode
12. Infinite scroll
13. Grouping
14. Aggregates & totals
15. Resize · reorder · visibility · sticky
16. Row selection
17. Editing, creation & deletion
18. Validation
19. Row reorder
20. Clipboard & cell ranges
21. Custom templates
22. Export & print
23. Internationalization
24. RTL
25. Theming
26. State persistence
27. Keyboard & accessibility
28. API reference
29. Known limitations

@inandu-solutions/grid-angular — User Manual

A standalone Angular data grid built from two components. No `NgModule`s, signal-based inputs and outputs, every feature opt-in, and a deliberately small dependency footprint.

1. Introduction

`@inandu-solutions/grid-angular` renders a `<table>` from a plain array of row objects. You compose it from two standalone components:

- `<inandu-grid>` — the grid itself; you bind `[data]` and turn features on with inputs.

- `<inandu-column>` — one per column, projected between the grid's tags. It renders nothing itself; the grid reads these as configuration.

DESIGN PRINCIPLES

- **Opt-in.** Out of the box you get a plain sortless, filterless table. Each capability is a single attribute.
- **The grid never mutates your data.** Edits, deletes, reorders and pastes are emitted as events — you decide whether and how to persist them.
- **Self-contained.** No dependency on Angular Material or any theming engine. Internal dependencies (`@angular/cdk` , `@ngx-translate/core` , `jspdf`) are implementation details you never configure.
- **Rows are identified by object reference.** There is no row-id concept — keep your row objects stable across re-renders (a static array or a signal you update immutably both qualify).

2. Installation

```
npm install @inandu-solutions/grid-angular
```

PEER DEPENDENCIES

`@angular/core` and `@angular/common` `^21.2.0` — you almost certainly already have them.

BUNDLED DEPENDENCIES (NOTHING TO CONFIGURE)

- `@angular/cdk` — virtual scroll.
- `@ngx-translate/core` — internal i18n; each grid instance gets its own isolated translation service.
- `jspdf` — PDF export, dynamically imported only when `exportPdf()` is actually called.
- `tslib` .

3. Quick start

Both components are standalone — add them to your component's `imports` :

```

import { Component } from '@angular/core';
import { InanduGridColumnComponent, InanduGridRow } from '@inandu-solutions,

@Component({
  selector: 'app-people',
  imports: [InanduGridColumnComponent, InanduGridRow],
  template: `
    <inandu-grid
      [data]="rows"
      [paging]="{ pageSize: 10 }"
      filter="true"
      lang="en">
      <inandu-column field="name" title="Name" sortable="true"></inandu-column>
      <inandu-column field="age" title="Age" type="number" sortable="true"></inandu-c
      <inandu-column field="joined" title="Joined" type="date" format="DD/MM/YYYY"></inanu
      <inandu-column field="active" title="Active" type="boolean" format="Yes|No"></inandu-c
    </inandu-grid>
  `,
})
export class PeopleComponent {
  rows: InanduGridRow[] = [
    { name: 'Ada Lovelace', age: 36, joined: new Date(2021, 2, 14), active: true },
    { name: 'Alan Turing', age: 41, joined: new Date(2019, 8, 1), active: false },
  ];
}

```

NOTE

Numeric string attributes such as `width="400"` and boolean-ish attributes such as `sortable="true"` / bare `sortable` / `filter="yes"` are coerced for you. Anything other than the literal string `"false"` counts as `"on"`.

4. Data & rows

`[data]` takes `InanduGridRow[]`, where `InanduGridRow` is just `Record<string, unknown>`. Bind an array or a signal's value:

```
<inandu-grid [data]="rows()"> ... </inandu-grid>
```

GENERIC ROW TYPING

`InanduGridComponent<T>` infers `T` from `[data]`, so every row-bearing output is typed as your row type instead of the untyped default:

```
interface Customer { id: number; name: string; }  
// rows: Customer[] → (rowSave)="save($event)" gives you { row: Customer, values: Partial<
```

This does *not* extend into `<inandu-column field="...">` — the field name is a plain string and is not checked against `T`.

CONTRACT

The grid consumes a plain array only. Fetching, HTTP, and persistence live in your app — handle the grid's output events and update your own array (and/or your backend) however you like.

5. Columns

Every column is one `<inandu-column>`. The only required input is `field`.

INPUT	TYPE	DEFAULT	PURPOSE
<code>field</code>	<code>string</code> (required)	—	Key read from each row object.
<code>title</code>	<code>string</code>	<code>''</code>	Header text. Falls back to <code>field</code> when empty.
<code>width</code>	<code>number</code>	<code>0</code>	Initial column width in px (0 = auto).
<code>order</code>	<code>number</code>	unset	0-based target slot. See below.

THE `ORDER` INPUT

`order` is a **target position, not a sort key**. Columns with an explicit `order` land at that 0-based slot (ties and out-of-range values fall back to declaration order); columns without one fill the remaining slots in declaration order. So `order="0"` / `order="1"` on two columns puts them first and second no matter where they appear in the template.

6. Cell types & formatting

`<inandu-column type="..." format="...">` controls how a raw cell value is displayed. `type` defaults to `'string'`. `null` / `undefined` always render as an empty string regardless of type.

`type="number"`

`format` is a `DecimalPipe`-style digits string: `'{minIntegerDigits}.{minFractionDigits}-{maxFractionDigits}'`. Thousands and decimal separators come from the app's `LOCALE_ID` — they are not part of `format`. Defaults to `'1.0-3'`.

```
<inandu-column field="price" type="number" format="1.2-2"></inandu-column>
// 1234.5 → "1,234.50" (en-US) / "1.234,50" (es-*)
```

`type="date"`

`format` is a **case-sensitive** token pattern using the field names from the ECMA-262 Date Time String Format — *not* Angular's `DatePipe` tokens. Any other character passes through literally. Defaults to `'YYYY-MM-DD'`.

TOKEN	MEANING
YYYY	Full year
MM	Month (01–12) — capital
DD	Day of month
HH	Hours (00–23)
mm	Minutes — lowercase
ss	Seconds
sss	Milliseconds

```
<inandu-column field="joined" type="date" format="DD/MM/YYYY HH:mm"></inandu-column>
// → "31/12/2025 23:59"
```

The raw value may be a `Date`, an ISO string, or a timestamp. A space-separated `"YYYY-MM-DD HH:mm"` is normalised to the ISO `T` form before parsing, so it renders consistently across browsers (Safari's date parser rejects the space form). Unparseable values fall back to `String(value)`.

`type="boolean"`

`format` is `'<truthy>|<falsy>'`, split on `|`. Defaults to `'true|false'`.

```
<inandu-column field="active" type="boolean" format="Sí|No"></inandu-column>
```

7. Sorting per column

`<inandu-column sortable="true">` renders a sort button in that header.

- **Plain click** — sorts ascending; a second click on the same column flips to descending; clicking a different column resets to ascending on the new one. There is no third "clear" click.
- **Shift-click** — appends the column as an additional, lower-priority sort criterion (or flips its direction if already active), building a multi-column sort. A small priority badge (1, 2, 3...) appears once two or more criteria are active.

Comparison: numeric subtraction for two numbers, `.getTime()` for two `Date`s, otherwise `localeCompare` following the app's `LOCALE_ID`. Nullish values sort first.

Programmatic control: `setSort(criteria)` / `clearSort()`. Any sort change resets pagination to page 1.

8. Filtering

Free-text search grid

`<inandu-grid filter="true">` adds a single search box above the header row. It matches the query as a substring against every visible column's *formatted* value (so searching "sí" matches a boolean column formatted `Sí|No`). `filterPlaceholder` overrides the placeholder text.

Per-column filters per column

`<inandu-column filter="yes">` adds a funnel button to that header opening a small popup. Several columns can have it at once; the filters combine with AND, and also AND with the free-text search if that is on. Only one popup is open at a time; a document click outside it closes it.

COLUMN TYPE	CONTROL	MATCHES
<code>string</code> (default)	text input	substring of the formatted value
<code>number</code>	min / max inputs	inclusive range on the raw number
<code>date</code>	from / to date pickers	inclusive range on the raw <code>Date</code> (<code>to</code> covers the whole day)
<code>boolean</code>	all / true / false select	exact equality (option labels come from the column's <code>format</code>)

The full pipeline is **data** → **free-text filter** → **column filters** → **sort** → **paginate**. Any filter change resets pagination to page 1.

9. Pagination grid

Bind the `paging` input to an `InanduGridPagingOptions` object. Pagination is off (every row renders) when `paging` is not bound at all; binding *any* object — even `{}` — turns it on.

```
readonly paging: InanduGridPagingOptions = { pageSize: 25 };  
  
<inandu-grid [data]="rows" [paging]="paging"> ... </inandu-grid>
```

FIELD	TYPE	DEFAULT
<code>pageSize</code>	<code>number</code>	<code>10</code>
<code>showFirstButton</code> ... <code>showLastButton</code>	<code>boolean</code> ×4	<code>true</code>
<code>firstLabel</code> ... <code>lastLabel</code>	<code>string</code> ×4	unset → built-in chevron icon

FIELD	TYPE	DEFAULT
<code>pageLabel</code>	<code>(page, totalPages) => string</code>	the resolved language's "Page X of Y"

Setting a `*Label` renders that string literally instead of the icon. Nav buttons carry translated `aria-label`s regardless of any custom glyph.

10. Virtual scroll grid

`virtualScroll` renders only the rows currently in view (plus a small runway), via `@angular/cdk`. It takes over from pagination entirely — the full sorted/filtered set is fed to the viewport and the pager footer hides itself.

```
<inandu-grid [data]="rows" virtualScroll="true" [height]="400"> ... </inandu-grid>
```

- The viewport needs a bounded height — it reuses the `height` input (falling back to `400`).
- `virtualRowHeight` is an optional px override. Left unset, the grid auto-measures its first rendered row.
- Works while grouped too (group headers are interleaved into the virtual list), assuming a uniform row height.

11. Server-side mode grid

When `serverSide` is on, the grid stops sorting, filtering and slicing locally — `data()` is trusted to be the current page, already processed by your server. Instead the grid emits an event whenever the user changes the relevant control, **including once on init** so you can drive the first fetch from it.

```
<inandu-grid
  [data]="pageRows()"
  serverSide="true"
  [totalItems]="total()"
  [loading]="loading()"
```



```

[error]="error()"
[paging]="{ pageSize: 50 }"
(sortChange)="onSort($event)"
(pageChange)="onPage($event)"
(filterChange)="onFilter($event)">
...
</inandu-grid>

```

INPUT / OUTPUT	MEANING
<code>totalItems</code>	Total row count across all pages — the pager needs it to compute page count. Falls back to <code>data().length</code> .
<code>loading</code>	Shows a loading row instead of data; disables the export toolbar.
<code>error</code>	Non-empty string → shows an error row (verbatim; translate it yourself). Wins over <code>loading</code> .
<code>(sortChange)</code>	<code>InanduGridSortCriterion[]</code> — ordered, index 0 first. Empty array = no sort.
<code>(pageChange)</code>	<code>{ page, pageSize }</code>
<code>(filterChange)</code>	<code>{ query, columnFilters }</code>

12. Infinite scroll grid

`infiniteScroll` is a no-op unless `virtualScroll` **and** `serverSide` are both on. As the user scrolls within `infiniteScrollThreshold` rows (default 10) of the end of the loaded data, the grid emits `(loadMore)` with `{ loadedCount }` — you fetch the next chunk and *append* it to `data()`. It fires at most once per distinct `data().length`.

13. Grouping per column

`<inandu-column groupable="true">` makes that header draggable onto a drop zone that appears at the top of the grid. Dropping it groups every row by that column's *formatted* value; dropping a different groupable column switches the grouping. Grouping bypasses pagination — every row shows, in groups, with a per-group header row.

Programmatic: `setGroupBy(field)` (validates against groupable columns) / `setGroupBy(undefined)` to clear.

14. Aggregates & totals

`<inandu-column aggregate="sum">` (`sum` · `avg` · `min` · `max` · `count`) drives two things from the same config:

- A **per-group subtotal** shown in each group header row while grouped.
- With grid-level `showTotals`, a **grand total** across the whole filtered/sorted set in a sticky footer row.

Non-numeric, blank, `null` and boolean cells are ignored by `sum` / `avg` / `min` / `max` (a genuine numeric string still counts); `count` counts rows. The aggregate kind is shown as a short symbol (`Σ` / `̄x` / `min` / `max` / `#`), not a translated word. `showTotals` is ignored while `virtualScroll` is on.

15. Resize · reorder · visibility · sticky

Resize opt-out

Every column is resizable by default. `<inandu-column resize="false">` pins the width. Dragging the handle at a header's edge **overrides** that column's initial `width` and persists across sort/filter/page changes.

Reorder opt-out

Every column is draggable to a new position by default. `<inandu-column reorder="false">` pins a column in place — other columns can still be dropped next to it, it just can't be picked up. A dropped column always lands immediately before the drop target.

Visibility toggle grid

`<inandu-grid columnToggle="true">` adds a toolbar button opening a checklist of columns.

`<inandu-column hideable="false">` keeps a column out of that list (it can never be hidden). Hiding

is refused once only one column is still visible. A hidden column is excluded from filters, editing, grouping *and* export output.

Sticky columns per column

`<inandu-column sticky="true">` freezes a column while the grid scrolls horizontally.
`stickySide="right"` freezes it to the right edge instead of the left. The selection checkbox column is always sticky-left when `selectable` is on.

LIMITATION

For predictable results, sticky columns should form a contiguous run from the first column (or the last, for `stickySide="right"`). A lone sticky column outside that run can visually overlap non-sticky columns mid-scroll — this is how CSS `position: sticky` works, not something the library detects.

16. Row selection grid

`<inandu-grid selectable="true">` prepends a checkbox column (header "select all" + one per row) and emits the live selection through `(selectionChange)` as `T[]`.

- Selection is tracked by object reference and is **not** cleared on sort/filter/page changes — rows keep their checked state as they move around.
- The header checkbox selects/clears the currently *visible* rows (the current page, or every group's rows while grouped / virtualized), and shows a tri-state `indeterminate`.
- Programmatic: `selectRows(rows)` / `deselectAll()` ; read `selectedRowsList()`.

Bulk delete: with `selectable` + `deletable`, a "Delete selected" toolbar button appears while at least one row is selected. It optionally confirms via `bulkDeleteConfirmMessage`, emits `(rowsDelete)` with the selected rows, then clears the selection.

17. Editing, creation & deletion

These share a trailing "actions" column. It renders when any column is `editable`, or the grid is `deletable` / `creatable`, or a `rowActionsTemplate` is provided.

Inline editing per column

`<inandu-column editable="true">`. Editing is per **row**: an "Edit" button per row switches every editable field in that row into a type-aware control at once and swaps in Save / Cancel. Only one row edits at a time; other rows' Edit and Delete buttons disable meanwhile.

```
<inandu-grid [data]="rows" (rowSave)="onSave($event)">
  <inandu-column field="name"  editable="true"></inandu-column>
  <inandu-column field="price" type="number" editable="true"></inandu-column>
</inandu-grid>

onSave(e: InanduGridRowSave) {
  // e.row  = the exact data() row reference (unchanged)
  // e.values = { name, price } parsed to real types
  this.rows.update(rs => rs.map(r => r === e.row ? { ...r, ...e.values } : r));
}
```

The grid **never** writes the edit back into `data()`. An emptied number or date field is omitted from `values` rather than committed as `0` / an invalid date.

Row creation grid

`creatable="true"` adds an "Add row" trigger at the top of the body (hidden while grouped). Save emits `(rowCreate)` with the parsed values — there's no row reference, the row doesn't exist yet.

Row deletion grid

`deletable="true"` adds a Delete button per row. With `deleteConfirmMessage` set, a native `window.confirm()` gates it. Emits `(rowDelete)` with the row reference; you filter it out of your own array.

18. Validation

Configured per column, checked once at save time (there is no live-as-you-type validation). The first failing rule's message is shown inline under the field and blocks the save; edit/create mode stays active so the user can fix it. Rules run in this fixed order:

1. `required` — fails on an empty text/number/date value (meaningless for `boolean`).

2. `min` / `max` — inclusive bounds on the parsed number (`number` columns only).
3. `pattern` — a `RegExp` source string tested against the raw string. A malformed pattern is treated as "no pattern", not an error.
4. `[validator]` — `(value, row) => string | null`. Runs after every built-in rule passes; `row` is the in-progress parsed values.
5. `[asyncValidator]` — `(value, row) => Promise<string | null>`. All fields' async checks run concurrently; Save / Cancel are disabled while any is pending.

```
<inandu-column field="code" editable="true"
  required="true" pattern="^[A-Z]{2}\d{4}$"></inandu-column>

<inandu-column field="qty" type="number" editable="true"
  min="1" [validator]="evenOnly"></inandu-column>

evenOnly = (v: unknown) => typeof v === 'number' && v % 2 ? 'Must be even' : null;
```

Built-in messages (`required` / `min` / `max` / `pattern`) are translated; a `validator` 's returned string is shown verbatim.

19. Row reorder grid

`rowReorder="true"` prepends a drag-handle column. Dragging a row emits `(rowOrderChange)` with the **complete** reordered array — you apply it back to your own `data()`. Ignored while grouped, virtualized, or in server-side mode. Meant to be used without an active sort (a re-sort immediately reverts the visual drag).

20. Clipboard & cell ranges

Clipboard grid

`clipboard="true"` enables Ctrl/Cmd+C / V while a data cell has keyboard focus. Copy puts the focused cell's formatted value on the clipboard; paste reads tab/newline-separated text (exactly what Excel/Sheets copy) and fans it out from the focused cell, emitting one `(cellsPaste)` array of `{ row, field, value }` — clipped at the grid edges, skipping non-editable columns. Values are

parsed per the target column's `type`. Works on the flat (non-grouped, non-virtualized) render path only.

Cell range selection grid

`cellRangeSelection="true"` — click-drag or shift-click across cells selects a rectangle, emitted through `cellRangeChange` as `{ rows, fields }` (or `undefined` when cleared). When `clipboard` is also on, Ctrl+C copies the whole range as TSV.

21. Custom templates

PER-COLUMN CELL & HEADER

```
<inandu-column field="status">
  <ng-template #cellTemplate let-value let-row="row">
    <span class="badge" [class.on]="value">{{ row.name }}</span>
  </ng-template>
</inandu-column>
```

Cell context: `$implicit` = raw value, `row`. Header context: `$implicit` = resolved title, `field`.

ROW ACTIONS

A `<ng-template let-row>` placed as a **direct child of** `<inandu-grid>` (not inside a column) renders after the built-in Edit/Delete in each existing row's actions cell.

22. Export & print grid

`exportable="true"` renders a toolbar above the table with four buttons: CSV, Excel, PDF, Print.

- **CSV** — `\r\n`-joined, with a UTF-8 BOM so Excel reads accented text correctly.
- **Excel** — a SpreadsheetML `.xls` (plain XML, opens natively in Excel / LibreOffice / Sheets, zero added dependency). Numeric columns keep a real number type. A binary `.xlsx` is a `@inandu-solutions/grid-pro` feature.
- **PDF** — a simple table drawn with `jspdf` (dynamically imported). Single-line cells, truncated with an ellipsis; header repeats per page.

- **Print** — opens a clean, isolated print document (not the live page) and calls `print()`.

All four operate on the **currently visible rows** — the current page, or every group's / all rows while grouped or virtualized — and on visible columns in their current order. Buttons disable together when there's nothing to export. The programmatic methods (`exportCsv()` , `exportExcel()` , `exportPdf()` , `printTable()`) are public.

23. Internationalization

`lang` is a BCP 47 tag (e.g. `lang="es-AR"`). Only the primary subtag is matched, so `es-AR` and `es-ES` both resolve to `es` . Built-in: **en, es, fr, it, zh**. Anything else falls back to the browser language, then English. Each grid instance has its own isolated translation service, so two grids on one page can show different languages.

OVERRIDING OR ADDING MESSAGES

```
<inandu-grid [customTranslations]="{
  en: { MsgNoData: 'Nothing to show' },
  pt: { MsgNoData: 'Nada para mostrar', MsgFirstPage: 'Primeira' }
}" lang="pt"> ... </inandu-grid>
```

Merged on top of the built-ins for that language (only the keys you supply change). A brand-new language code works as a fresh entry — this is the only way to add a whole new language.

24. RTL

`dir="rtl"` (or `"auto"`) sets a plain `dir` attribute on the root. It composes with `lang` rather than replacing it — `lang="ar"` alone still renders left-to-right unless `dir="rtl"` is also set. Layout mirrors via CSS logical properties; the pager chevron icons are mirrored too. A consumer-supplied pager label string is never mirrored.

25. Theming

`theme` is an optional plain string. Omitted, the grid is unstyled-plain. Set it and the grid adds an `inandu-theme-{value}` class. Three presets ship (pure CSS, no engine): `material`, `dark`, `minimal`.

```
<inandu-grid [data]="rows" theme="material"> ... </inandu-grid>
```

Any other value works too — supply a matching `.inandu-theme-<name>` rule in your own stylesheet.

STYLING HOOKS

Every structural element carries a stable, rule-free class you can target from your global stylesheet:

`.inandu-grid` (root), `.inandu-column` (`<col>` and `<th>`), `.inandu-row` (each body `<tr>`), and many feature-specific ones (`.inandu-pager-button`, `.inandu-sort-button`, `.inandu-filter-input`, ...).

```
inandu-grid .inandu-row:nth-child(even) { background: #fafafa; }
```

26. State persistence `grid`

Give the grid a non-empty `stateKey` string and it persists column widths, order, visibility, the active sort, and all filters to `localStorage` — restoring them the next time a grid with the same key mounts (a reload, or navigating away and back). All access is wrapped in `try/catch`, so private-browsing / quota errors are silently ignored.

```
<inandu-grid [data]="rows" stateKey="customers-grid"> ... </inandu-grid>
```

NOTE

Give two different grids distinct keys, or they overwrite each other's saved state.

27. Keyboard & accessibility

- Header and data cells carry `role="columnheader"` / `role="gridcell"`; rows carry `role="row"`.

- A roving tabIndex means one cell is a tab stop at a time. Arrow keys / Home / End move focus between cells.
- All action buttons (pager, sort, filter toggle, row actions, export) carry a translated `aria-label` and a `title`.
- The select-all checkbox reflects a real `indeterminate` state.

28. API reference

<inandu-grid> inputs

INPUT	TYPE	DEFAULT
data	T[]	[]
id	string	' ' (used as the export filename prefix)
width / height	number	0
paging	InanduGridPagingOptions	unset → pagination off
filter	boolean	false
filterPlaceholder	string	translated default
virtualScroll	boolean	false
virtualRowHeight	number	auto-measured
serverSide	boolean	false
totalItems	number	data().length
loading / error	boolean / string	false / ' '
infiniteScroll	boolean	false
infiniteScrollThreshold	number	10
selectable	boolean	false

INPUT	TYPE	DEFAULT
<code>columnToggle</code>	<code>boolean</code>	<code>false</code>
<code>rowReorder</code>	<code>boolean</code>	<code>false</code>
<code>clipboard</code>	<code>boolean</code>	<code>false</code>
<code>cellRangeSelection</code>	<code>boolean</code>	<code>false</code>
<code>showTotals</code>	<code>boolean</code>	<code>false</code>
<code>exportable</code>	<code>boolean</code>	<code>false</code>
<code>creatable</code>	<code>boolean</code>	<code>false</code>
<code>deletable</code>	<code>boolean</code>	<code>false</code>
<code>deleteConfirmMessage</code>	<code>string</code>	<code>''</code> (no prompt)
<code>bulkDeleteConfirmMessage</code>	<code>string</code>	<code>''</code>
<code>stateKey</code>	<code>string</code>	<code>''</code> (no persistence)
<code>lang</code>	<code>string</code> (BCP 47)	browser → <code>en</code>
<code>customTranslations</code>	<code>InanduGridCustomTranslations</code>	unset
<code>dir</code>	<code>'ltr' 'rtl' 'auto'</code>	<code>'ltr'</code>
<code>theme</code>	<code>string</code>	<code>''</code> (no theme class)

<inandu-grid> outputs

OUTPUT	PAYLOAD
<code>selectionChange</code>	<code>T[]</code>
<code>rowSave</code>	<code>InanduGridRowSave<T></code> — <code>{ row, values }</code>
<code>rowCreate</code>	<code>InanduGridNewRowValues<T></code> — parsed values
<code>rowDelete</code>	<code>T</code> — the row reference

OUTPUT	PAYLOAD
<code>rowsDelete</code>	<code>T[]</code>
<code>rowOrderChange</code>	<code>T[]</code> — the full reordered array
<code>cellsPaste</code>	<code>InanduGridCellPaste<T>[]</code>
<code>cellRangeChange</code>	<code>InanduGridCellRangeSelection<T> undefined</code>
<code>sortChange</code>	<code>InanduGridSortCriterion[]</code> (server-side)
<code>pageChange</code>	<code>InanduGridPageState</code> (server-side)
<code>filterChange</code>	<code>InanduGridFilterState</code> (server-side)
<code>loadMore</code>	<code>InanduGridLoadMoreEvent</code> — <code>{ loadedCount }</code>

`<inandu-grid>` public methods

METHOD	EFFECT
<code>setFilterQuery(q)</code>	Set the free-text query.
<code>setSort(criteria)</code> / <code>clearSort()</code>	Replace / clear the multi-column sort.
<code>setGroupBy(field undefined)</code>	Group / ungroup (validated against groupable columns).
<code>selectRows(rows)</code> / <code>deselectAll()</code>	Replace / clear the selection.
<code>goToPage(n)</code>	Jump to a 1-based page (clamped).
<code>exportCsv()</code> / <code>exportExcel()</code> / <code>exportPdf()</code> / <code>printTable()</code>	Trigger the corresponding export.
<code>selectedRowslist()</code>	Read-only signal of the current selection.

`<inandu-column>` inputs

INPUT	TYPE	DEFAULT
<code>field</code>	<code>string</code> — required	—

INPUT	TYPE	DEFAULT
<code>title</code>	<code>string</code>	<code>field</code>
<code>width</code>	<code>number</code>	<code>0</code>
<code>order</code>	<code>number</code>	unset (declaration order)
<code>type</code>	<code>'string' 'number' 'boolean' 'date'</code>	<code>'string'</code>
<code>format</code>	<code>string</code>	per type
<code>sortable</code>	<code>boolean</code>	<code>false</code>
<code>filter</code>	<code>boolean</code>	<code>false</code>
<code>groupable</code>	<code>boolean</code>	<code>false</code>
<code>resize</code>	<code>boolean</code>	<code>true</code>
<code>reorder</code>	<code>boolean</code>	<code>true</code>
<code>hideable</code>	<code>boolean</code>	<code>true</code>
<code>sticky</code>	<code>boolean</code>	<code>false</code>
<code>stickySide</code>	<code>'left' 'right'</code>	<code>'left'</code>
<code>aggregate</code>	<code>'' 'sum' 'avg' 'min' 'max' 'count'</code>	<code>''</code>
<code>editable</code>	<code>boolean</code>	<code>false</code>
<code>required</code>	<code>boolean</code>	<code>false</code>
<code>min</code> / <code>max</code>	<code>number</code>	unset
<code>pattern</code>	<code>string</code> (RegExp source)	<code>''</code>
<code>[validator]</code>	<code>InanduColumnValidator</code>	unset
<code>[asyncValidator]</code>	<code>InanduColumnAsyncValidator</code>	unset

Exported types

```
import type {
  InanduGridRow,           // Record<string, unknown>
  InanduGridPagingOptions,
  InanduGridRowSave,       // { row, values }
  InanduGridNewRowValues,
  InanduGridSortCriterion, // { field, direction }
  InanduGridPageState,
  InanduGridFilterState,
  InanduGridLoadMoreEvent,
  InanduGridCellPaste,
  InanduGridCellRangeSelection,
  InanduGridCustomTranslations,
  InanduRowActionsContext,
  SortDirection,           // 'asc' | 'desc'
  InanduColumnType,
  InanduColumnValidator,
  InanduColumnAsyncValidator,
  InanduColumnStickySide,
  InanduColumnAggregate,
  InanduCellTemplateContext,
  InanduHeaderTemplateContext,
} from '@inandu-solutions/grid-angular';
```

29. Known limitations

- **Sticky columns** outside a contiguous leading (or trailing) run can visually overlap mid-scroll — inherent CSS behaviour.
- **Virtual scroll + grouping** assumes a uniform row height, including group-header rows.
- **PDF export** is single-line only — cells are truncated with an ellipsis, no wrapping.
- **Excel export** is SpreadsheetML `.xls`. A styled binary `.xlsx` is a `@inandu-solutions/grid-pro` feature.
- **Row reorder** with an active sort visually reverts on the next re-render.
- **Master-detail rows** are not part of the MIT core.
- **Safari < 16**: sticky table cells with `border-collapse: collapse` can be unreliable — current Safari is fine.

